

LAPORAN PRATIKUM
STRUKTUR DATA
“SEARCHING (BFS dan DFS)”

PEKAN KE-9

disusun Oleh:

Aisyah Najmiatul Fauziah

NIM 2511533030

Dosen Pengampu: DR. WAHYUDI, S.T, M.T

Asisten Pratikum: Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data dengan judul “**Searching (BFS dan DFS)**” pada Pekan ke-9 ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum yang telah dilakukan.

Praktikum ini membahas tentang algoritma pencarian pada struktur data graf, yaitu **Breadth First Search (BFS)** dan **Depth First Search (DFS)**. Kedua algoritma tersebut merupakan metode penting dalam proses penelusuran simpul pada graf yang banyak digunakan dalam berbagai bidang ilmu komputer, seperti pencarian jalur, kecerdasan buatan, jaringan komputer, dan pengolahan data.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan laporan di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai algoritma BFS dan DFS.

Padang, 10 Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II.....	2
PEMBAHASAN	2
2.1 Praktikum “BTree_2511533030”	2
2.2 Praktikum “Node_2511533030”.....	2
2.3 Praktikum “BTreeDriver_2511533030”.....	2
2.4 Pratikum “GraphTraversal_2511533030”.....	3
BAB III.....	3
KESIMPULAN.....	3
3.1 Kesimpulan.....	3

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data graf merupakan salah satu struktur data yang digunakan untuk merepresentasikan hubungan antar objek. Dalam implementasinya, sering kali diperlukan proses pencarian atau penelusuran terhadap simpul-simpul yang terdapat pada graf untuk menemukan informasi tertentu atau menentukan jalur yang dapat ditempuh.

Untuk melakukan proses penelusuran tersebut, terdapat beberapa algoritma pencarian yang umum digunakan, di antaranya **Breadth First Search (BFS)** dan **Depth First Search (DFS)**. BFS melakukan pencarian dengan menelusuri simpul berdasarkan tingkat kedekatan dari simpul awal, sedangkan DFS melakukan pencarian dengan menelusuri satu cabang hingga kedalaman tertentu sebelum berpindah ke cabang lainnya.

Pemahaman terhadap kedua algoritma ini sangat penting karena menjadi dasar dalam penyelesaian berbagai permasalahan komputasi, seperti pencarian rute, traversal graf, analisis jaringan, dan berbagai aplikasi lainnya. Oleh karena itu, praktikum ini dilaksanakan agar mahasiswa dapat memahami konsep serta implementasi BFS dan DFS dalam pemrograman.

1.2 Tujuan Pratikum

1. Memahami konsep dasar pencarian (searching) pada struktur data graf.
2. Memahami cara kerja algoritma Breadth First Search (BFS).
3. Memahami cara kerja algoritma Depth First Search (DFS).
4. Mengimplementasikan algoritma BFS dan DFS menggunakan bahasa pemrograman Java.
5. Menganalisis hasil penelusuran simpul yang diperoleh dari kedua algoritma tersebut.
6. Membandingkan karakteristik dan perbedaan BFS dengan DFS dalam proses pencarian pada graf.

1.3 Manfaat Pratikum

1. Menambah pemahaman mengenai struktur data graf dan proses traversal graf.
2. Melatih kemampuan mahasiswa dalam mengimplementasikan algoritma BFS dan DFS ke dalam program Java.
3. Meningkatkan kemampuan berpikir logis dan analitis dalam menyelesaikan permasalahan pencarian data.
4. Memberikan pengalaman praktis dalam penggunaan struktur data seperti **Queue** pada BFS dan **Stack/Recursion** pada DFS.

BAB II

PEMBAHASAN

2.1 Praktikum “BTree_2511533030”

Langkah Kerja ;

1. Buat Package dan Class baru
2. Buat kode berikut

```
package pekan9_2511533030;

public class BTree_2511533030 {
    private Node_2511533030 root_3030;
    private Node_2511533030 currentNode_3030;

    public BTree_2511533030() {
        root_3030 = null;
    }

    public boolean search_3030(int data_3030) {
        return search_3030(root_3030, data_3030);
    }

    private boolean search_3030(Node_2511533030 node_3030, int data_3030) {
        if (node_3030.getData_3030() == data_3030)
            return true;

        if (node_3030.getLeft_3030() != null)
            if (search_3030(node_3030.getLeft_3030(), data_3030))
                return true;

        if (node_3030.getRight_3030() != null)
            if (search_3030(node_3030.getRight_3030(), data_3030))
                return true;

        return false;
    }

    public void printInorder_3030() {
        root_3030.printInorder_3030(root_3030);
    }

    public void printPreorder_3030() {
        root_3030.printPreorder_3030(root_3030);
    }

    public void printPostorder_3030() {
        root_3030.printPostorder_3030(root_3030);
    }

    public Node_2511533030 getRoot_3030() {
        return root_3030;
    }

    public boolean isEmpty() {
        return root_3030 == null;
    }

    public int countNodes_3030() {
        return countNodes_3030(root_3030);
    }
}
```

```

    }

    private int countNodes_3030(Node_2511533030 node_3030) {
        int count_3030 = 1;

        if (node_3030 == null) {
            return 0;
        } else {
            count_3030 += countNodes_3030(node_3030.getLeft_3030());
            count_3030 += countNodes_3030(node_3030.getRight_3030());
            return count_3030;
        }
    }

    public void print_3030() {
        root_3030.print();
    }

    public Node_2511533030 getCurrent_3030() {
        return currentNode_3030;
    }

    public void setCurrent_3030(Node_2511533030 node_3030) {
        this.currentNode_3030 = node_3030;
    }

    public void setRoot_3030(Node_2511533030 root_3030) {
        this.root_3030 = root_3030;
    }
}

```

Gambar 2.1 kode program “BTree_2511533030”

Penjelasan Kode :

2.1.1 Deklarasi Variabel dan Konstruktor

- Bagian ini digunakan untuk mendeklarasikan node akar (root_3030) dan node aktif (currentNode_3030).
- Konstruktor digunakan untuk membuat Binary Tree baru dengan kondisi awal kosong (root = null).

```

package pekan9_2511533030;
public class BTree_2511533030 {
    private Node_2511533030 root_3030;
    private Node_2511533030 currentNode_3030;

    public BTree_2511533030() {
        root_3030 = null;
    }
}

```

2.1.2 Method Pencarian Data (Search)

- Method ini berfungsi untuk mencari data pada Binary Tree menggunakan teknik rekursif.
- Pencarian dilakukan mulai dari root, kemudian menelusuri subtree kiri dan subtree kanan hingga data ditemukan atau seluruh node telah diperiksa.

```

public boolean search_3030(int data_3030) {
    return search_3030(root_3030, data_3030);
}
private boolean search_3030(Node_2511533030 node_3030, int data_3030) {
    if (node_3030.getData_3030() == data_3030)
        return true;
    if (node_3030.getLeft_3030() != null)
        if (search_3030(node_3030.getLeft_3030(), data_3030))
            return true;
    if (node_3030.getRight_3030() != null)
        if (search_3030(node_3030.getRight_3030(), data_3030))
            return true;
    return false;
}

```

2.1.3 Method Traversal Tree

- Method traversal digunakan untuk menampilkan isi Binary Tree dengan tiga cara:
 - **Inorder** : Kiri → Root → Kanan
 - **Preorder** : Root → Kiri → Kanan
 - **Postorder** : Kiri → Kanan → Root

```

public void printInorder_3030() {
    root_3030.printInorder_3030(root_3030);
}
public void printPreorder_3030() {
    root_3030.printPreorder_3030(root_3030);
}
public void printPostorder_3030() {
    root_3030.printPostorder_3030(root_3030);
}
public Node_2511533030 getRoot_3030() {
    return root_3030;
}

```

2.1.4 Method Pemeriksaan dan Penghitungan Node

- Method **isEmpty()** digunakan untuk memeriksa apakah Binary Tree kosong.
- Sedangkan method **countNodes_3030()** digunakan untuk menghitung jumlah seluruh node pada Binary Tree secara rekursif.

```

public boolean isEmpty() {
    return root_3030 == null;
}
public int countNodes_3030() {
    return countNodes_3030(root_3030);
}
private int countNodes_3030(Node_2511533030 node_3030) {
    int count_3030 = 1;

    if (node_3030 == null) {
        return 0;
    } else {
        count_3030 += countNodes_3030(node_3030.getLeft_3030());
        count_3030 += countNodes_3030(node_3030.getRight_3030());
        return count_3030;
    }
}

```

```
}  
}
```

2.1.5 Method Menampilkan Tree

- Method ini digunakan untuk menampilkan struktur atau isi Binary Tree dengan memanggil method **print()** yang terdapat pada kelas **Node_2511533030**.

```
public void print_3030() {  
    root_3030.print();  
}
```

2.1.6 Method Getter dan Setter

- Method getter dan setter digunakan untuk mengambil serta mengubah nilai root dan current node pada Binary Tree.

```
public Node_2511533030 getCurrent_3030() {  
    return currentNode_3030;  
}  
public void setCurrent_3030(Node_2511533030 node_3030) {  
    this.currentNode_3030 = node_3030;  
}  
public void setRoot_3030(Node_2511533030 root_3030) {  
    this.root_3030 = root_3030;  
}  
}
```

2.2 Praktikum “Node_2511533030”

Langkah Kerja :

1. Buat Class baru
2. Buat kode berikut

```
package pekan9_2511533030;  
public class Node_2511533030 {  
    int data_3030;  
    Node_2511533030 left_3030;  
    Node_2511533030 right_3030;  
    public Node_2511533030(int data_3030) {  
        this.data_3030 = data_3030;  
        left_3030 = null;  
        right_3030 = null;  
    }  
    public void setLeft_3030(Node_2511533030 node_3030) {  
        if (left_3030 == null)  
            left_3030 = node_3030;  
    }  
    public void setRight_3030(Node_2511533030 node_3030) {  
        if (right_3030 == null)  
            right_3030 = node_3030;  
    }  
    public Node_2511533030 getLeft_3030() {  
        return left_3030;  
    }  
    public Node_2511533030 getRight_3030() {  
        return right_3030;  
    }  
    public int getData_3030() {  
        return data_3030;  
    }  
}
```



```

    }
    public void setData_3030(int data_3030) {
        this.data_3030 = data_3030;
    }
    void printPreorder_3030(Node_2511533030 node_3030) {
        if (node_3030 == null)
            return;
        System.out.print(node_3030.data_3030 + " ");
        printPreorder_3030(node_3030.left_3030);
        printPreorder_3030(node_3030.right_3030);
    }
    void printPostorder_3030(Node_2511533030 node_3030) {
        if (node_3030 == null)
            return;
        printPostorder_3030(node_3030.left_3030);
        printPostorder_3030(node_3030.right_3030);
        System.out.print(node_3030.data_3030 + " ");
    }
    void printInorder_3030(Node_2511533030 node_3030) {
        if (node_3030 == null)
            return;
        printInorder_3030(node_3030.left_3030);
        System.out.print(node_3030.data_3030 + " ");
        printInorder_3030(node_3030.right_3030);
    }
    public String print() {
        return this.print("", true, "");
    }
    public String print(String prefix_3030, boolean isTail_3030, String sb_3030) {
        if (right_3030 != null) {
            right_3030.print(prefix_3030 + (isTail_3030 ? "| " : " "), false,
sb_3030);
        }
        System.out.println(prefix_3030 + (isTail_3030 ? "\\-- " : "/--") +
data_3030);
        if (left_3030 != null) {
            left_3030.print(prefix_3030 + (isTail_3030 ? " " : "| "), true,
sb_3030);
        }
        return sb_3030;
    }
}

```

Gambar 2.2 kode program “Node_2511533030”

Langkah Kerja :

2.2.1 Deklarasi Variabel dan Konstruktor

- Bagian ini digunakan untuk mendefinisikan atribut yang dimiliki oleh setiap node pada Binary Tree :
 - **data_3030** menyimpan nilai yang terdapat pada node.
 - **left_3030** menyimpan alamat anak kiri.
 - **right_3030** menyimpan alamat anak kanan.
- Konstruktor digunakan untuk membuat node baru dengan nilai tertentu. Saat node dibuat, anak kiri dan kanan masih kosong (null).

```

int data_3030;
Node_2511533030 left_3030;
Node_2511533030 right_3030;

```

```

public Node_2511533030(int data_3030) {
    this.data_3030 = data_3030;
    left_3030 = null;
    right_3030 = null;
}

```

2.2.2 Method Setter dan Getter

- Method-method ini digunakan untuk mengakses dan mengubah data pada node.
- Fungsi Setter :
 - **setLeft_3030()** menghubungkan node dengan anak kiri.
 - **setRight_3030()** menghubungkan node dengan anak kanan.
 - **setData_3030()** mengubah nilai data pada node.
- Fungsi Getter :
 - **getLeft_3030()** mengambil anak kiri.
 - **getRight_3030()** mengambil anak kanan.
 - **getData_3030()** mengambil nilai data node.

```

public void setLeft_3030(Node_2511533030 node_3030) {
    if (left_3030 == null)
        left_3030 = node_3030;
}
public void setRight_3030(Node_2511533030 node_3030) {
    if (right_3030 == null)
        right_3030 = node_3030;
}
public Node_2511533030 getLeft_3030() {
    return left_3030;
}
public Node_2511533030 getRight_3030() {
    return right_3030;
}
public int getData_3030() {
    return data_3030;
}
public void setData_3030(int data_3030) {
    this.data_3030 = data_3030;
}
}

```

2.2.3 Method Traversal Tree

- Bagian ini digunakan untuk menampilkan isi Binary Tree dengan tiga teknik traversal.
- Preorder :
 - Urutan traversal :
Root → Kiri → Kanan
 Node induk dicetak terlebih dahulu, kemudian anak kiri dan anak kanan.
- Inorder :
 - Urutan Traversal :
Kiri → Root → Kanan
 Anak kiri dicetak terlebih dahulu, kemudian root dan anak kanan.
- Postorder :
 - Urutan traversal :
Kiri → Kanan → Root

Anak kiri dan kanan dicetak terlebih dahulu, kemudian root.

- Seluruh traversal dilakukan menggunakan teknik rekursif.

```
void printPreorder_3030(Node_2511533030 node_3030) {
    if (node_3030 == null)
        return;
    System.out.print(node_3030.data_3030 + " ");
    printPreorder_3030(node_3030.left_3030);
    printPreorder_3030(node_3030.right_3030);
}
void printPostorder_3030(Node_2511533030 node_3030) {
    if (node_3030 == null)
        return;
    printPostorder_3030(node_3030.left_3030);
    printPostorder_3030(node_3030.right_3030);
    System.out.print(node_3030.data_3030 + " ");
}
void printInorder_3030(Node_2511533030 node_3030) {
    if (node_3030 == null)
        return;
    printInorder_3030(node_3030.left_3030);
    System.out.print(node_3030.data_3030 + " ");
    printInorder_3030(node_3030.right_3030);
}
```

2.2.4 Method Menampilkan Struktur Binary Tree

- Method ini digunakan untuk menampilkan bentuk Binary Tree secara visual.
- Proses pencetakan dilakukan dengan:
 - Menampilkan subtree kanan terlebih dahulu.
 - Menampilkan node saat ini.
 - Menampilkan subtree kiri.
- Dari tampilan tersebut dapat diketahui bahwa:
 - Root = 20
 - Anak kanan = 30
 - Anak kiri = 10

```
public String print() {
    return this.print("", true, "");
}
public String print(String prefix_3030, boolean isTail_3030, String sb_3030) {
    if (right_3030 != null) {
        right_3030.print(prefix_3030 + (isTail_3030 ? "| " : " "), false,
sb_3030);
    }
    System.out.println(prefix_3030 + (isTail_3030 ? "\\-- " : "/--") +
data_3030);
    if (left_3030 != null) {
        left_3030.print(prefix_3030 + (isTail_3030 ? " " : "| "), true,
sb_3030);
    }
    return sb_3030;
}
}
```

2.3 Praktikum “BTreeDriver_2511533030”

Langkah Kerja :

1. Buat Class Baru
2. Buat kode berikut

```
package pekan9_2511533030;

public class BTreeDriver_2511533030 {
    public static void main(String[] args) {
        // Membuat pohon
        BTree_2511533030 tree_3030 = new BTree_2511533030();
        System.out.print("Jumlah Simpul awal pohon: ");
        System.out.println(tree_3030.countNodes_3030());
        // Menambahkan simpul data 1
        Node_2511533030 root_3030 = new Node_2511533030(1);
        // Menjadikan simpul 1 sebagai root
        tree_3030.setRoot_3030(root_3030);
        System.out.println("Jumlah simpul jika hanya ada root: ");
        System.out.println(tree_3030.countNodes_3030());
        Node_2511533030 node2 = new Node_2511533030(2);
        Node_2511533030 node3 = new Node_2511533030(3);
        Node_2511533030 node4 = new Node_2511533030(4);
        Node_2511533030 node5 = new Node_2511533030(5);
        Node_2511533030 node6 = new Node_2511533030(6);
        Node_2511533030 node7 = new Node_2511533030(7);
        Node_2511533030 node8 = new Node_2511533030(8);
        Node_2511533030 node9 = new Node_2511533030(9);
        root_3030.setLeft_3030(node2);
        node2.setLeft_3030(node4);
        node2.setRight_3030(node5);
        node4.setRight_3030(node8);
        root_3030.setRight_3030(node3);
        node3.setLeft_3030(node6);
        node3.setRight_3030(node7);
        node6.setLeft_3030(node9);
        // Set root
        tree_3030.setCurrent_3030(tree_3030.getRoot_3030());
        System.out.println("Menampilkan simpul terakhir: ");
        System.out.println(tree_3030.getCurrent_3030().getData_3030());
        System.out.println("Jumlah simpul setelah simpul 7 ditambahkan");
        System.out.println(tree_3030.countNodes_3030());
        System.out.println("InOrder: ");
        tree_3030.printInorder_3030();
        System.out.println("\nPreorder: ");
        tree_3030.printPreorder_3030();
        System.out.println("\nPostorder: ");
        tree_3030.printPostorder_3030();
        System.out.println("\nMenampilkan simpul dalam bentuk pohon");
        tree_3030.print_3030();
    }
}
```

Gambar 2.3 kode program “BTreeDriver_2511533030”

3. Didapatkan Output Sebagai Berikut

```

<terminated> BTreeDriver_2511533030 [Java Application] C:\Users\AISYAH NA
Jumlah simpul setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder:
8 4 5 2 9 6 7 3 1
Menampilkan simpul dalam bentuk pohon
|
| /--7
| /--3
| | \--6
| | \--9
| \--1
| | /--5
| | \--2
| | | /--8
| | | \--4

```

Gambar 2.3 output program “BTreeDriver_2511533030”

Penjelasan Kode :

2.3.1 Pembuatan Objek Binary Tree

- Bagian ini digunakan untuk membuat objek Binary Tree baru yang masih kosong.
- Setelah itu program menghitung jumlah simpul yang ada pada pohon.
- Karena belum terdapat node, maka jumlah simpul yang ditampilkan adalah 0.

```

// Membuat pohon
BTree_2511533030 tree_3030 = new BTree_2511533030();
System.out.print("Jumlah Simpul awal pohon: ");
System.out.println(tree_3030.countNodes_3030());

```

2.3.2 Pembuatan Root Tree

- Bagian ini membuat node dengan nilai 1 dan menjadikannya sebagai root dari Binary Tree.
- Setelah root ditambahkan, jumlah simpul pada pohon menjadi 1.

```

// Menambahkan simpul data 1
Node_2511533030 root_3030 = new Node_2511533030(1);
// Menjadikan simpul 1 sebagai root
tree_3030.setRoot_3030(root_3030);
System.out.println("Jumlah simpul jika hanya ada root: ");
System.out.println(tree_3030.countNodes_3030());

```

2.3.3 Pembuatan Seluruh Node

- bagian ini digunakan untuk membuat node-node yang akan dimasukkan ke dalam Binary Tree dengan nilai 2 sampai 9.

```

Node_2511533030 node2 = new Node_2511533030(2);
Node_2511533030 node3 = new Node_2511533030(3);
Node_2511533030 node4 = new Node_2511533030(4);
Node_2511533030 node5 = new Node_2511533030(5);
Node_2511533030 node6 = new Node_2511533030(6);
Node_2511533030 node7 = new Node_2511533030(7);
Node_2511533030 node8 = new Node_2511533030(8);
Node_2511533030 node9 = new Node_2511533030(9);

```

2.3.4 Pembentukan Struktur Binary Tree

- Bagian ini digunakan untuk menghubungkan node-node
- Setiap node dihubungkan menggunakan method setLeft_3030() dan setRight_3030().

```

root_3030.setLeft_3030(node2);
node2.setLeft_3030(node4);

```

```

node2.setRight_3030(node5);
node4.setRight_3030(node8);
root_3030.setRight_3030(node3);
node3.setLeft_3030(node6);
node3.setRight_3030(node7);
node6.setLeft_3030(node9);

```

2.3.5 Menentukan Current Node

- Bagian ini digunakan untuk menjadikan root sebagai current node.
- Kemudian data dari current node ditampilkan. Karena current node adalah root, maka nilai yang ditampilkan adalah **1**.

```

// Set root
tree_3030.setCurrent_3030(tree_3030.getRoot_3030());
System.out.println("Menampilkan simpul terakhir: ");
System.out.println(tree_3030.getCurrent_3030().getData_3030());

```

2.3.6 Menghitung Jumlah Simpul

- Method **countNodes_3030()** digunakan untuk menghitung jumlah seluruh simpul yang terdapat pada Binary Tree.
- Pada pohon yang telah dibentuk terdapat **9 simpul**, yaitu node 1 sampai node 9.

```

System.out.println("Jumlah simpul setelah simpul 7 ditambahkan");
System.out.println(tree_3030.countNodes_3030());

```

2.3.7 Menampilkan Traversal Binary Tree

- Bagian ini digunakan untuk menampilkan hasil traversal Binary Tree.
- Traversal Inorder :
Kiri → Root → Kanan
- Traversal Preorder
Root → Kiri → Kanan
- Traversal Postorder
Kiri → Kanan → Root

```

System.out.println("InOrder: ");
tree_3030.printInorder_3030();
System.out.println("\nPreorder: ");
tree_3030.printPreorder_3030();
System.out.println("\nPostorder: ");

```

2.3.8 Menampilkan Struktur Pohon

- Bagian ini digunakan untuk menampilkan Binary Tree dalam bentuk visual sehingga hubungan antara root, anak kiri, dan anak kanan dapat terlihat dengan jelas.

```

System.out.println("\nMenampilkan simpul dalam bentuk pohon");
tree_3030.print_3030();
}
}

```

2.4 Pratikum “GraphTraversal_2511533030”

Langkah Kerja :

1. Buat Class Baru
2. Buat Kode Berikut

```

package pekan9_2511533030;

```

```

import java.util.*;

public class GraphTraversal_2511533030 {
    private Map<String, List<String>> graph_3030 = new HashMap<>();
    // Menambahkan edge (graf tak berarah)
    public void addEdge(String node1, String node2) {
        graph_3030.putIfAbsent(node1, new ArrayList<>());
        graph_3030.putIfAbsent(node2, new ArrayList<>());
        graph_3030.get(node1).add(node2);
        graph_3030.get(node2).add(node1);
    }
    // Menampilkan graf awal
    public void printGraph() {
        System.out.println("Graf Awal (Adjacency List):");
        for (String node : graph_3030.keySet()) {
            System.out.print(node + " -> ");
            List<String> neighbors = graph_3030.get(node);
            System.out.println(String.join(", ", neighbors));
        }
        System.out.println();
    }
    // DFS rekursif
    public void dfs(String start) {
        Set<String> visited = new HashSet<>();
        System.out.println("Penelusuran DFS:");
        dfsHelper(start, visited);
        System.out.println();
    }
    private void dfsHelper(String current, Set<String> visited) {
        if (visited.contains(current)) return;
        visited.add(current);
        System.out.print(current + " ");
        for (String neighbor : graph_3030.getDefault(current, new
ArrayList<>())) {
            dfsHelper(neighbor, visited);
        }
    }
    // BFS iteratif
    public void bfs(String start) {
        Set<String> visited = new HashSet<>();
        Queue<String> queue = new LinkedList<>();
        queue.add(start);
        visited.add(start);
        System.out.println("Penelusuran BFS:");
        while (!queue.isEmpty()) {
            String current = queue.poll();
            System.out.print(current + " ");
            for (String neighbor : graph_3030.getDefault(current, new
ArrayList<>())) {
                if (!visited.contains(neighbor)) {
                    queue.add(neighbor);
                    visited.add(neighbor);
                }
            }
        }
        System.out.println();
    }
    // Main
    public static void main(String[] args) {

```

```

GraphTraversal_2511533030 graph_3030 = new GraphTraversal_2511533030();
// Contoh graf: A-B, A-C, B-D, B-E
graph_3030.addEdge("A", "B");
graph_3030.addEdge("A", "C");
graph_3030.addEdge("B", "D");
graph_3030.addEdge("B", "E");
// Cetak graf awal
System.out.println("Graf Awal adalah: ");
graph_3030.printGraph();
// Lakukan penelusuran
graph_3030.dfs("A");
graph_3030.bfs("A");
}

```

Gambar 2.4 kode program “GraphTraversal_2511533030”

3. Didapatkan Output Sebagai Berikut

```

<terminated> GraphTraversal_2511533030 [Java Application] C:\Users\AISYAH NAJMIATUL F
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E

```

Gambar 2.4 output program “GraphTraversal_2511533030”

Penjelasan Kode :

2.4.1 Deklarasi Struktur Graf dan Penambahan Edge

- Bagian ini digunakan untuk membangun struktur graf menggunakan **Adjacency List**.
- Variabel graph_3030 berfungsi menyimpan simpul beserta daftar tetangganya.
- Method **addEdge()** digunakan untuk menambahkan hubungan antar simpul.
- Karena graf yang digunakan merupakan **graf tak berarah (undirected graph)**, maka setiap hubungan disimpan pada kedua simpul yang terhubung.

```

private Map<String, List<String>> graph_3030 = new HashMap<>();
// Menambahkan edge (graf tak berarah)
public void addEdge(String node1, String node2) {
    graph_3030.putIfAbsent(node1, new ArrayList<>());
    graph_3030.putIfAbsent(node2, new ArrayList<>());
    graph_3030.get(node1).add(node2);
    graph_3030.get(node2).add(node1);
}

```

2.4.2 Menampilkan Graf

- Bagian ini digunakan untuk menampilkan isi graf dalam bentuk **Adjacency List**.

- Program akan membaca seluruh simpul yang terdapat pada graf kemudian menampilkan semua tetangga yang terhubung dengan simpul tersebut.
- Tampilan ini memudahkan pengguna untuk melihat hubungan antar simpul dalam graf sebelum proses traversal dilakukan.

```
// Menampilkan graf awal
public void printGraph() {
    System.out.println("Graf Awal (Adjacency List):");
    for (String node : graph_3030.keySet()) {
        System.out.print(node + " -> ");
        List<String> neighbors = graph_3030.get(node);
        System.out.println(String.join(", ", neighbors));
    }
    System.out.println();
}
```

2.4.3 Traversal Menggunakan Depth First Search (DFS)

- Bagian ini mengimplementasikan algoritma **Depth First Search (DFS)** untuk melakukan penelusuran graf.
- DFS bekerja dengan cara menelusuri satu jalur hingga mencapai simpul terdalam terlebih dahulu sebelum kembali ke simpul sebelumnya dan melanjutkan pencarian ke cabang lain.
- Untuk mencegah simpul yang sama dikunjungi berulang kali, digunakan struktur data **Set** bernama **visited**.
- Proses traversal dilakukan secara rekursif melalui method **dfsHelper()**.
- Jika graf dimulai dari simpul A, maka urutan traversal yang dihasilkan adalah **A, B, D, E, C**.

```
// DFS rekursif
public void dfs(String start) {
    Set<String> visited = new HashSet<>();
    System.out.println("Penelusuran DFS:");
    dfsHelper(start, visited);
    System.out.println();
}

private void dfsHelper(String current, Set<String> visited) {
    if (visited.contains(current)) return;
    visited.add(current);
    System.out.print(current + " ");
    for (String neighbor : graph_3030.getDefault(current, new
ArrayList<>())) {
        dfsHelper(neighbor, visited);
    }
}
```

2.4.4 Traversal Menggunakan Breadth First Search (BFS)

- Bagian ini mengimplementasikan algoritma **Breadth First Search (BFS)**.
- Berbeda dengan DFS, BFS melakukan penelusuran berdasarkan tingkat kedekatan simpul.
- Simpul yang berada pada level yang sama akan dikunjungi terlebih dahulu sebelum melanjutkan ke level berikutnya.
- Untuk mengatur urutan kunjungan digunakan struktur data **Queue (antrian)**. Simpul awal dimasukkan ke dalam antrian, kemudian diproses satu per satu hingga seluruh simpul yang terhubung berhasil dikunjungi.

- Pada graf yang digunakan, traversal BFS menghasilkan urutan **A, B, C, D, E**.

```
// BFS iteratif
public void bfs(String start) {
    Set<String> visited = new HashSet<>();
    Queue<String> queue = new LinkedList<>();
    queue.add(start);
    visited.add(start);
    System.out.println("Penelusuran BFS:");
    while (!queue.isEmpty()) {
        String current = queue.poll();
        System.out.print(current + " ");
        for (String neighbor : graph_3030.getDefault(current, new
ArrayList<>())) {
            if (!visited.contains(neighbor)) {
                queue.add(neighbor);
                visited.add(neighbor);
            }
        }
    }
    System.out.println();
}
```

2.4.5 Program Utama (Main)

- Bagian ini merupakan program utama yang berfungsi untuk menguji seluruh operasi graf yang telah dibuat.
- Program membuat sebuah graf dengan simpul A, B, C, D, dan E, kemudian menghubungkannya menggunakan method addEdge().
- Setelah graf terbentuk, program menampilkan graf dalam bentuk Adjacency List dan melakukan traversal menggunakan algoritma DFS dan BFS dengan titik awal simpul A.
- Hasil traversal yang ditampilkan menunjukkan perbedaan cara kerja kedua algoritma dalam menelusuri graf.

```
// Main
public static void main(String[] args) {
    GraphTraversal_2511533030 graph_3030 = new GraphTraversal_2511533030();
    // Contoh graf: A-B, A-C, B-D, B-E
    graph_3030.addEdge("A", "B");
    graph_3030.addEdge("A", "C");
    graph_3030.addEdge("B", "D");
    graph_3030.addEdge("B", "E");
    // Cetak graf awal
    System.out.println("Graf Awal adalah: ");
    graph_3030.printGraph();
    // Lakukan penelusuran
    graph_3030.dfs("A");
    graph_3030.bfs("A");
}
```

BAB II

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data **Binary Tree** dan **Graf** merupakan struktur data yang sangat penting dalam pengolahan data. Pada implementasi Binary Tree, berbagai operasi seperti pembentukan pohon, perhitungan jumlah simpul, pencarian data, serta traversal menggunakan metode **Inorder, Preorder, dan Postorder** dapat dilakukan dengan baik menggunakan teknik rekursif.

Selain itu, pada implementasi struktur data Graf, proses penelusuran simpul dapat dilakukan menggunakan algoritma **Depth First Search (DFS)** dan **Breadth First Search (BFS)**. Algoritma DFS melakukan penelusuran hingga ke simpul terdalam terlebih dahulu sebelum kembali ke simpul sebelumnya, sedangkan BFS melakukan penelusuran berdasarkan tingkat kedekatan simpul dengan memanfaatkan struktur data Queue.

Melalui praktikum ini, mahasiswa dapat memahami konsep dasar Binary Tree dan Graf, mengetahui perbedaan cara kerja algoritma traversal yang digunakan, serta mampu mengimplementasikannya menggunakan bahasa pemrograman Java.

Pengetahuan ini menjadi dasar yang penting dalam mempelajari berbagai algoritma dan struktur data yang lebih kompleks pada bidang ilmu komputer.

DAFTAR PUSTAKA

Lafore, Robert. ***Data Structures and Algorithms in Java***. 2nd Edition.
Indianapolis: Sams Publishing, 2003.

TUGAS PRATIKUM

1. Kode Program

```
package pekan9_2511533030;
import javax.swing.*;
import java.awt.*;
import java.util.*;
public class PetaRumahSakit_2511533030 extends JFrame {
    static Map<String, ArrayList<String>> graph_3030 = new LinkedHashMap<>();
    static Map<String, Point> pos_3030 = new HashMap<>();
    String[] lokasi_3030 = {
        "Parkir",
        "Lobby",
        "IGD",
        "Farmasi",
        "Radiologi",
        "Laboratorium",
        "Poli Umum",
        "Poli Gigi",
        "Ruang Inap",
        "ICU",
        "Kantin",
        "Masjid"
    };
    Set<String> visited_3030 = new LinkedHashSet<>();
    ArrayList<String> path_3030 = new ArrayList<>();
    String start_3030 = "";
    String goal_3030 = "";
    String metode_3030 = "";
    JComboBox<String> cbStart_3030;
    JComboBox<String> cbGoal_3030;
    JTextArea hasil_3030;
    JPanel graphPanel_3030 = new JPanel() {
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            displayGraph_3030((Graphics2D) g);
        }
    };
    public PetaRumahSakit_2511533030() {
        setTitle("Pencarian Jalur Rumah Sakit Menggunakan BFS dan DFS");
        setSize(900, 700);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        buatGraph_3030();
        buatGUI_3030();
        setVisible(true);
    }
    void buatGraph_3030() {
        for (String s : lokasi_3030) {
            graph_3030.put(s, new ArrayList<>());
        }
        tambahEdge_3030("Parkir", "Lobby");
        tambahEdge_3030("Lobby", "IGD");
        tambahEdge_3030("Lobby", "Farmasi");
        tambahEdge_3030("Lobby", "Poli Umum");
        tambahEdge_3030("IGD", "ICU");
        tambahEdge_3030("IGD", "Laboratorium");
    }
}
```

```

    tambahEdge_3030("Farmasi", "Kantin");
    tambahEdge_3030("Farmasi", "Ruang Inap");
    tambahEdge_3030("Poli Umum", "Laboratorium");
    tambahEdge_3030("Poli Umum", "Poli Gigi");
    tambahEdge_3030("Poli Gigi", "Radiologi");
    tambahEdge_3030("Laboratorium", "Radiologi");
    tambahEdge_3030("Radiologi", "Ruang Inap");
    tambahEdge_3030("ICU", "Ruang Inap");
    tambahEdge_3030("Ruang Inap", "Masjid");
    tambahEdge_3030("Kantin", "Masjid");
    tambahEdge_3030("Poli Umum", "Farmasi");
    pos_3030.put("Parkir", new Point(400, 60));
    pos_3030.put("Lobby", new Point(400, 140));
    pos_3030.put("IGD", new Point(180, 240));
    pos_3030.put("Farmasi", new Point(400, 240));
    pos_3030.put("Poli Umum", new Point(620, 240));
    pos_3030.put("Laboratorium", new Point(180, 360));
    pos_3030.put("Radiologi", new Point(400, 360));
    pos_3030.put("Poli Gigi", new Point(620, 360));
    pos_3030.put("ICU", new Point(180, 500));
    pos_3030.put("Ruang Inap", new Point(400, 500));
    pos_3030.put("Kantin", new Point(620, 500));
    pos_3030.put("Masjid", new Point(400, 620));
}
void tambahEdge_3030(String a, String b) {
    graph_3030.get(a).add(b);
    graph_3030.get(b).add(a);
}
void buatGUI_3030() {
    JPanel atas = new JPanel();
    cbStart_3030 = new JComboBox<>(lokasi_3030);
    cbGoal_3030 = new JComboBox<>(lokasi_3030);
    JButton bfs = new JButton("BFS");
    JButton dfs = new JButton("DFS");
    JButton reset = new JButton("RESET");
    atas.add(new JLabel("Start"));
    atas.add(cbStart_3030);
    atas.add(new JLabel("Goal"));
    atas.add(cbGoal_3030);
    atas.add(bfs);
    atas.add(dfs);
    atas.add(reset);
    add(atas, BorderLayout.NORTH);
    graphPanel_3030.setBackground(Color.WHITE);
    add(graphPanel_3030, BorderLayout.CENTER);
    hasil_3030 = new JTextArea(6, 20);
    hasil_3030.setEditable(false);
    hasil_3030.setFont(new Font("Arial", Font.PLAIN, 14));
    add(new JScrollPane(hasil_3030), BorderLayout.SOUTH);
    bfs.addActionListener(e -> BFS_3030());
    dfs.addActionListener(e -> DFS_3030());
    reset.addActionListener(e -> resetGraph_3030());
}
void BFS_3030() {
    visited_3030.clear();
    path_3030.clear();
    metode_3030 = "BFS";
    start_3030 = cbStart_3030.getSelectedItem().toString();
    goal_3030 = cbGoal_3030.getSelectedItem().toString();
}

```

```

Queue<String> q = new LinkedList<>();
Map<String, String> parent = new HashMap<>();
q.add(start_3030);
visited_3030.add(start_3030);
parent.put(start_3030, null);
boolean found = false;
while (!q.isEmpty()) {
    String current = q.poll();
    if (current.equals(goal_3030)) {
        found = true;
        break;
    }
    for (String next : graph_3030.get(current)) {
        if (!visited_3030.contains(next)) {
            visited_3030.add(next);
            parent.put(next, current);
            q.add(next);
        }
    }
}

if (found) {
    String temp = goal_3030;
    while (temp != null) {
        path_3030.add(0, temp);
        temp = parent.get(temp);
    }
    displayPath_3030();
}

void DFS_3030() {
    visited_3030.clear();
    path_3030.clear();
    metode_3030 = "DFS";
    start_3030 = cbStart_3030.getSelectedItem().toString();
    goal_3030 = cbGoal_3030.getSelectedItem().toString();
    dfsRekursif_3030(start_3030);
    displayPath_3030();
}

boolean dfsRekursif_3030(String node) {
    visited_3030.add(node);
    path_3030.add(node);
    if (node.equals(goal_3030)) {
        return true;
    }
    for (String next : graph_3030.get(node)) {
        if (!visited_3030.contains(next)) {
            if (dfsRekursif_3030(next)) {
                return true;
            }
        }
    }
}

path_3030.remove(path_3030.size() - 1);
return false;
}

void displayPath_3030() {
    hasil_3030.setText(
        "Metode : " + metode_3030 +

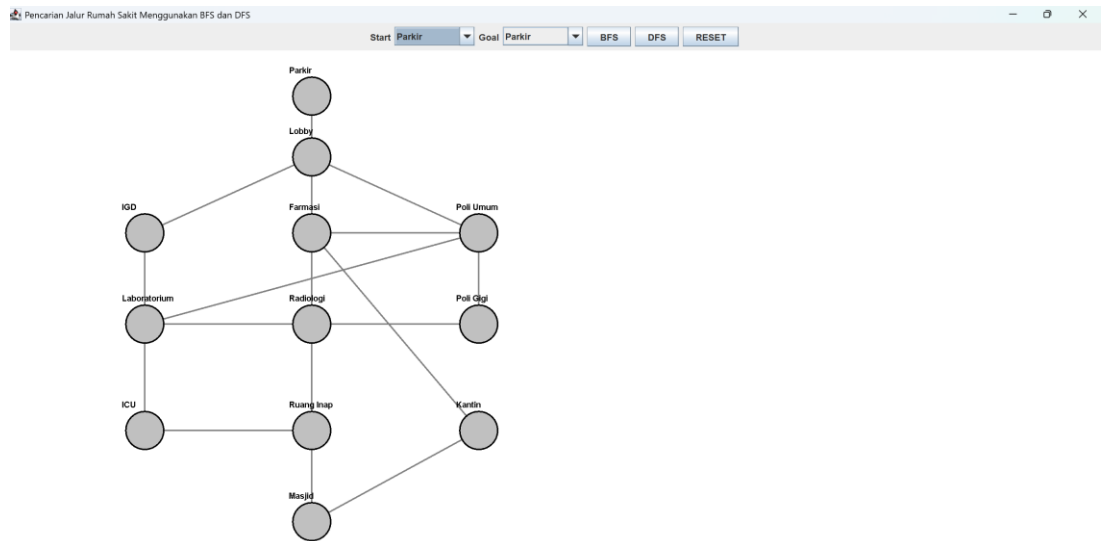
```

```

        "\nJalur : " + path_3030 +
        "\nNode Dikunjungi : " + visited_3030 +
        "\nJumlah Node Dieksplorasi : " + visited_3030.size()
    );
    graphPanel_3030.repaint();
}
void resetGraph_3030() {
    visited_3030.clear();
    path_3030.clear();
    hasil_3030.setText("");
    graphPanel_3030.repaint();
}
void displayGraph_3030(Graphics2D g_3030) {
    g_3030.setStroke(new BasicStroke(2));
    for (String a_3030 : graph_3030.keySet()) {
        for (String b_3030 : graph_3030.get(a_3030)) {
            if (a_3030.compareTo(b_3030) < 0) {
                Point p1_3030 = pos_3030.get(a_3030);
                Point p2_3030 = pos_3030.get(b_3030);
                g_3030.setColor(Color.GRAY);
                g_3030.drawLine(p1_3030.x, p1_3030.y, p2_3030.x, p2_3030.y);
            }
        }
    }
    for (String node_3030 : lokasi_3030) {
        Point p_3030 = pos_3030.get(node_3030);
        if (node_3030.equals(start_3030))
            g_3030.setColor(Color.GREEN);
        else if (node_3030.equals(goal_3030))
            g_3030.setColor(Color.RED);
        else if (path_3030.contains(node_3030))
            g_3030.setColor(Color.YELLOW);
        else if (visited_3030.contains(node_3030))
            g_3030.setColor(Color.CYAN);
        else
            g_3030.setColor(Color.LIGHT_GRAY);
        g_3030.fillOval(p_3030.x - 25, p_3030.y - 25, 50, 50);
        g_3030.setColor(Color.BLACK);
        g_3030.drawOval(p_3030.x - 25, p_3030.y - 25, 50, 50);
        g_3030.setFont(new Font("Arial", Font.BOLD, 11));
        g_3030.drawString(node_3030, p_3030.x - 30, p_3030.y - 30);
    }
}
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new PetaRumahSakit_2511533030();
    });
}
}

```


2. Ouput Program



3. Penjelasan

- **Deskripsi peta yang dibuat.**

Peta yang dibuat adalah peta sederhana Rumah Sakit yang terdiri dari beberapa lokasi penting yang sering dikunjungi pasien dan pengunjung. Lokasi-lokasi tersebut direpresentasikan sebagai simpul (node) dalam graph, sedangkan jalur penghubung antar lokasi direpresentasikan sebagai sisi (edge).

Node yang digunakan yaitu:

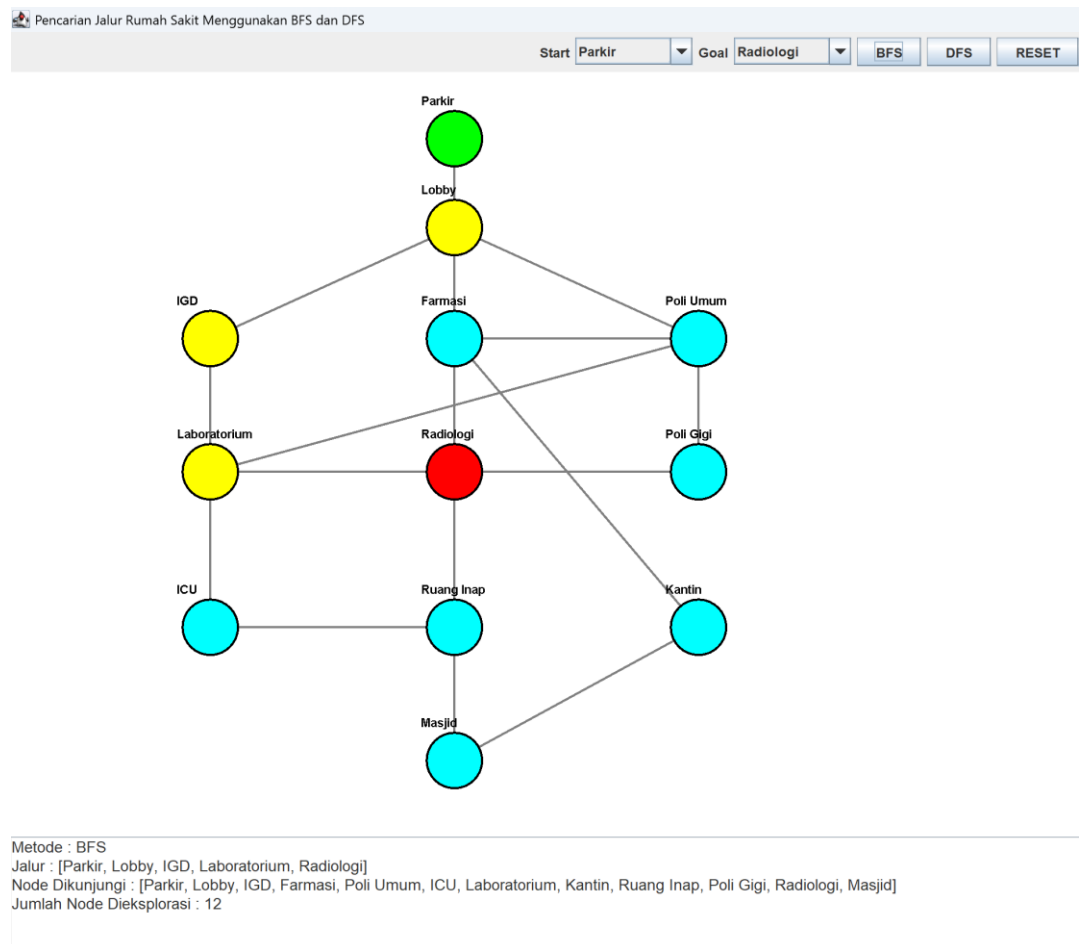
- Parkir
- Lobby
- IGD
- Farmasi
- Radiologi
- Laboratorium
- Poli Umum
- Poli Gigi
- Ruang Inap
- ICU
- Kantin
- Masjid

Graph ini digunakan untuk mencari jalur dari lokasi awal (Start) menuju lokasi tujuan (Goal) menggunakan algoritma BFS dan DFS.

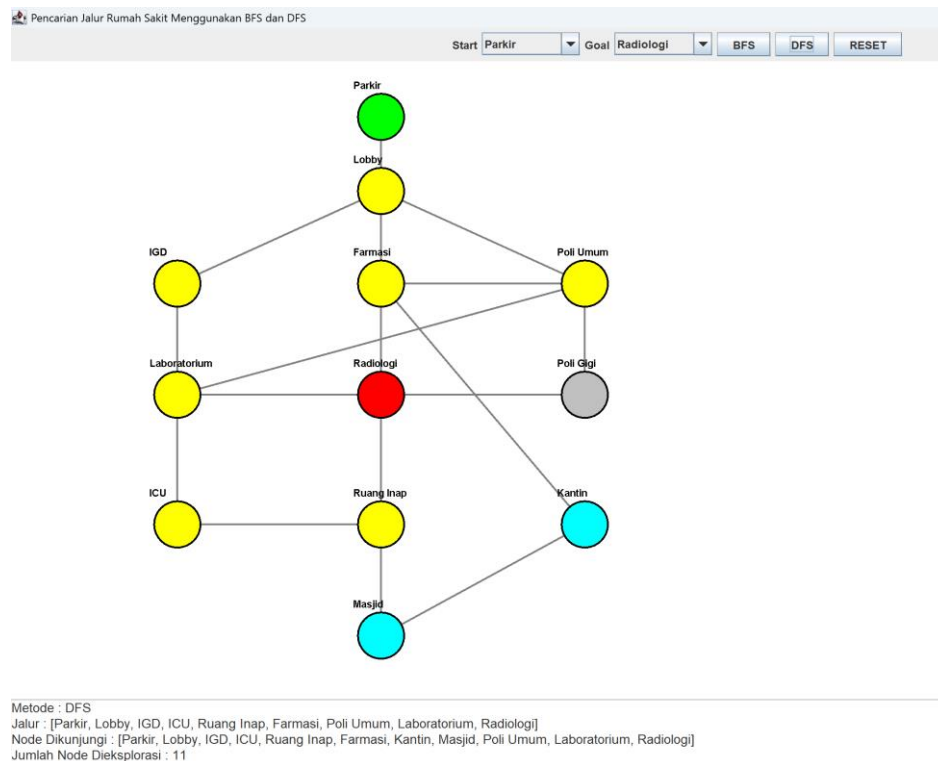
- Jumlah node dan edge
 - Jumlah Node (Vertex) : 12 Node
 - Jumlah Edge (Sisi) : 17 Edge
 - Daftar Edge
 - Parkir – Lobby
 - Lobby – IGD

- Lobby – Farmasi
- Lobby – Poli Umum
- IGD – ICU
- IGD – Laboratorium
- Farmasi – Kantin
- Farmasi – Ruang Inap
- Poli Umum – Laboratorium
- Poli Umum – Poli Gigi
- Poli Gigi – Radiologi
- Laboratorium – Radiologi
- Radiologi – Ruang Inap
- ICU – Ruang Inap
- Ruang Inap – Masjid
- Kantin – Masjid
- Poli Umum – Farmasi

- Hasil BFS



- Hasil DFS



- Kesimpulan perbandingan BFS dan DFS\

Berdasarkan hasil pengujian, algoritma BFS (Breadth First Search) dan DFS (Depth First Search) memiliki cara kerja yang berbeda.

 - BFS melakukan pencarian secara melebar (per level) sehingga mampu menemukan jalur terpendek menuju tujuan.
 - DFS melakukan pencarian secara mendalam terlebih dahulu hingga menemukan tujuan.
 - BFS biasanya mengunjungi lebih banyak node karena memeriksa semua kemungkinan pada setiap level.
 - DFS cenderung menggunakan memori lebih sedikit, tetapi jalur yang ditemukan belum tentu merupakan jalur terpendek.
 - Pada pengujian ini BFS menghasilkan jalur terpendek dari Parkir ke Masjid, sedangkan DFS menemukan jalur yang berbeda sesuai urutan pencarian node.

Kesimpulannya, BFS lebih cocok digunakan ketika tujuan utama adalah mendapatkan jalur terpendek, sedangkan DFS lebih cocok digunakan untuk pencarian yang tidak terlalu memperhatikan panjang jalur dan ingin melakukan penelusuran secara mendalam terlebih dahulu.